

백엔드에서 쌓은 판단을 AI Native 개발 방식으로 구조화합니다

Java와 Kotlin으로 이벤트·배치·검색·캐시 시스템을 설계하고 운영해 왔습니다. 그 과정에서 쌓은 판단을 에이전트·스킬·평가로 구조화해 실제 개발 흐름에서 재사용합니다.

서민재 · Rex Seo

AI-Native
Backend Engineer ckdekrn88@gmail.com

GitHub Velog

LinkedIn 이력서

경력기술서

9 · 6 · 8

내부 이벤트 구조

이벤트 9종이 멈춘 위치를 6개 상태로 남기고, 업무별 수동 복구 전략 8개를 연결했습니다.

9 · 500→100→20

피드 랭킹 단계

9개 소스를 병렬 조회하고 1차 랭킹 500건, 2차 랭킹 100건, 기본 페이지 20건으로 좁혔습니다.

13개 시나리오

포인트 동시성 계약

포인트 API 9개의 연속 재요청과 3·5·10개 동시 요청에서 잔액·원장 상태·후속 이벤트까지 확인했습니다.

15 + 32

AI 개발 자산

agent 15개와 skill 32개를 설치·검증 가능한 구조로 묶고 Codex 탐색 32/32를 확인했습니다.

01

POINT PLATFORM · EVENT RELIABILITY

발행 실패와 처리 실패, 같은 방식으로 복구할 수 없었습니다.

포인트 차감이나 회수가 늦게 실패하면 정산 확인과 고객 응대도 함께 늦어집니다. 실패 지점과 복구 대상을 이벤트 레코드에 남겨, 로그를 처음부터 추적하지 않고 멈춘 업무부터 다시 실행할 수 있게 했습니다.

담당

이벤트 상태·수동 복구·정체
알림 설계 및 구현

구조

내부 이벤트 9종, 상태 6개,
타입별 복구 전략 8개

저장

PostgreSQL
internal_event_record ·
JSONB payload

관측

IN_PROGRESS 15분 초과 건을
타입별로 집계해 Slack 전송

상황

‘실패’ 하나로 묶자, 원인을 찾는 일부터 다시 시작해야 했습니다.

메시지를 보내지 못한 경우와 소비자가 처리 중 멈춘 경우는 확인할 데이터도, 재시도할 위치도 다릅니다. 상태가 같으면 운영자가 로그와 원장을 일일이 맞춰 봐야 했습니다.

판단

멈춘 지점을 기록하고, 복구 로직은 업무별로 나눴습니다.

생성부터 복구 불가 실패까지 6개 상태를 두었습니다. 포인트 차감·회수·만료 알림처럼 조건이 다른 8개 작업은 저장된 payload로 각 업무 서비스를 다시 호출합니다.

멈춘 위치에서 다시 시작하는 복구 흐름

생성부터 처리, 수동 복구와 정체 감지까지

— 호출 — 비동기 — 복구

01 정상 처리

업무 요청과 처리 상태를 기록한 뒤 Kafka로 전달합니다.

업무 요청

API · Admin · Batch

차감 · 회수 · 사용 · 만료

기록

상태 기록

internal_event_record
InternalEventCreator
CREATED · JSONB payload

전달

KAFKA

내부 이벤트 topic 9개

발행 실패는 SEND_FAILED로 분리

처리

업무 처리

Listener 9개 · EventProcessor

IN_PROGRESS → 완료 · 실패

02 수동 복구

Kafka에 다시 넣지 않고 실패한 업무만 실행합니다.

복구 대상 조회

type · status로 검색

저장된 payload 복원

선택

복구 전략

Retry Strategy 8개

이벤트 타입별 업무 분기

재실행

DOMAIN SERVICE

업무 서비스 직접 호출

차감 · 회수 · 사용 · 알림

03 정체 감지

15분 넘게 처리 중인 이벤트를 Slack으로 알립니다.

QUERYDSL

IN_PROGRESS 15분 초과

추적 대상 이벤트 8종

집계

SLACK

이벤트 타입별 요약

운영 채널로 건수 전송

달라진 운영

전송 실패와 처리 실패를 서로 다른 상태로 기록합니다. 처리 중 15분 넘게 멈춘 이벤트는 Slack으로 알리고, 저장된 payload로 실패한 업무만 다시 실행합니다.

세부 구현과 예외 처리

- 내부 이벤트 9종을 생성, 발행 실패, 처리 중, 완료, 처리 실패, 복구 불가 실패의 6개 상태로 기록했습니다.
- 수동 복구 API가 타입과 상태로 레코드를 찾고 저장된 payload로 8개 전략 중 하나를 실행하도록 구성했습니다.
- Kafka에 다시 넣지 않고 타입별 업무 서비스를 직접 호출해 복구 범위를 실패한 업무로 제한했습니다.
- IN_PROGRESS 상태로 15분 이상 멈춘 추적 대상 8종만 조회해 타입별 건수를 Slack에 전송했습니다.
- 상태 전환 타이밍 오류, 잘못된 상태 검사, 이벤트 발송 실패 로깅 누락을 수정했습니다.
- 배치 재시도와 프록시 기반 재시도 호출 문제를 수정하고 포인트 선택 복구 경로를 보강했습니다.

배치는 멈춘 위치를, 동시 요청은 끝난 뒤의 상태를 확인했습니다.

원장 소멸·만료 알림·아카이브는 작업마다 다시 시작할 위치가 다릅니다. 포인트 거래와 예산 차감은 동시에 실행된 뒤 잔액과 원장 상태가 맞아야 합니다.

담당	처리 단위	보관 대상	동시성
원장 소멸 페이지 전환·아카이브 배치 구현·만료 알림 이벤트 개선	전체 적재 대신 페이지 단위 반복 처리	point_core 6개·point_service 4개 archive 테이블	9개 API · 13개 시나리오 · 3·5·10개 동시 요청

상황

세 Job이 같은 데이터를 만진다고 같은 방식으로 돌릴 수는 없었습니다.

소멸은 잔액과 원장을 바꾸고, 만료 안내는 알림을 쌓아 발송하며, 아카이브는 참조 순서대로 테이블을 옮깁니다. 전체 적재 방식은 데이터가 늘수록 실패 범위도 함께 커졌습니다.

판단

배치에는 재시작 기준을, 동시 요청에는 결과 불변식을 뒀습니다.

소멸·알림·아카이브는 진행 위치를 남겼습니다. 포인트 거래와 예산 차감은 락 획득 여부뿐 아니라 경쟁이 끝난 뒤 잔액·원장 상태·이벤트 수가 맞는지 확인했습니다.

세 배치, 세 가지 재시작 기준

같은 원장을 다뤄도 다시 시작할 기준은 서로 다릅니다.

— 순차 처리 - - - 후속 이벤트

01 소멸 처리 PointExpireJob	02 만료 알림 PointExpireAlarmJob	03 원장 보관 LedgerArchiveJob
대상 조회 pointUserId 기준 페이지 만료 원장 · 회원 매핑 TRANSACTION 유료·무료 잔액과 원장 PointExpireAppService 소멸 결과 expire_notification_record 후속 알림의 입력	PARTITION origin site별 Step beforeStep에서 대상 분리 500건씩 조회 기업 회원 필터 · PENDING 적재 PointExpireAlarmAppService KAFKA POINT_EXPIRATION_ALARM_SEND 회원 이메일과 발송 대상 전달 WORKER 메일 전송 · SUCCESS 발송 상태 반영	PAGING READER 보관 대상 원장 삭제한 뒤 0페이지부터 반복 APPLICATION operator 10개를 순서대로 실행 LedgerArchiveAppService POINT_CORE · POINT_SERVICE archive 테이블 10개 원장 · 상태 · 거래 · 매핑
재시작 기준	재시작 기준	재시작 기준
페이지	발송 상태	테이블 순서

락 키와 해제 시점에 따라 달라지는 세 가지 경쟁 결과

중복 차단·거래 정합성·어뷰징 분기의 세 경로로 나뉘었습니다.

9개 API 발급 · 거래 · 복구 · 예산 차감	13개 시나리오 같은 락 키 · 다른 거래 참조값 · 유지 시간 경계	3 · 5 · 10개 스레드 연속 재요청은 별도 타임라인으로 검증
01 검증 대상 API마다 락 키와 해제 시점은 달라도 경쟁 후 상태는 설명할 수 있어야 합니다. 1회용 키 중복 발급 차단 포인트 적립 잔액 · 한도 포인트 선정 무료 · 유료 잔액 포인트 사용 원장 · 선정 상태 선점 복구 잔액 · 복구 상태 포인트 환불 원장 · 환불 상태 사용 중단 원장 · 중단 상태 사용 재개 원장 · 재개 상태 클릭 예산 차감 · 어뷰징 이벤트		
경쟁 후 검증 기준 성공 건수 · 잔액 · 상태 · 이벤트		
02 중복 발급 억제 동일 사용자 · 락 유지 시간 10개 스레드 같은 사용자의 키 발급 요청 같은 식별자로 동시에 진입 차단 기대 결과 동시 요청 10건 · 생성된 키 1개 같은 사용자에게 키가 중복 생성되지 않음 락 유지 시간 경계 경과 후 재요청 허용	03 거래 순서와 정합성 동일 사용자 · 서로 다른 거래 참조값 3개 스레드 API별 서로 다른 거래 정보 3건 적립부터 사용 중단·재개까지 7개 거래 API 순차 처리 기대 결과 3건 모두 완료 성공 여부와 함께·상태를 함께 확인 불변식 잔액 · 원장 · 업무 상태	04 클릭 어뷰징 분기 복합 요청 식별자 · 락 유지 시간 5개 스레드 고유 요청과 중복 요청 복합 요청 식별자로 중복 여부를 구분 판별 기대 결과 고유 요청 5건 · 중복 요청 1건 처리 중복 4건은 대체 처리·어뷰징 이벤트로 전환 후속 처리 차감 · 대체 처리 · 이벤트
잔액 무료·유료 잔액과 적립 한도 누적액	원장·업무 상태 사용 · 복구 · 환불 · 중단 · 재개	후속 이벤트 차감 · 대체 처리 · 어뷰징 이벤트 수
CI 느린 테스트의 실행 경계 DisableSlowTest가 붙은 테스트는 기본 실행에 포함하고, skip.slow.test=true일 때 제외합니다. 실행 정책 기본 실행 · skip.slow.test=true 제외		

정합성과 회원 식별 처리

- 만료 대상 수집, 날짜 계산, 기업 회원 필터, 만료 알림 레코드 일괄 저장 순서를 검증했습니다.
 - point_core 6개와 point_service 4개 테이블의 아카이브 흐름과 테스트를 추가했습니다.
- 같은 락 키와 서로 다른 거래 참조값, 같은 예산을 돌려싼 동시 요청을 각각 검증했습니다.
 - 유료·무료 원장 혼합 사용 케이스를 별도로 검증했습니다.
 - 회원-워크스페이스 매핑, master/member 분리, 회원 탈퇴 이벤트, globalUserRef 중복 처리를 보강했습니다.

03

FEED & SEARCH · SERVING / RETRIEVAL

캐시는 얼마나 빨리 읽느냐보다 언제 무효화하느냐가 더 중요했습니다.

정상 스크롤에서는 후보를 재사용하고, 좌표·필터·로그인 상태가 바뀌면 Snapshot·FPR·SPR 후보 캐시만 무효화했습니다.

후보 조회	랭킹	캐시	분기
OpenSearch 9개 소스 병렬 실행, 소스별 최대 200건	FPR 500 → SPR/Re-Rank 100 → 페이지 20건	Snapshot · FPR · SPR · User Vector, TTL 30분	요청 지문 변경, guest → member, Redis 장애

상황	판단
조건이 바뀐 뒤에도 기존 후보를 쓰면 빠르지만 틀린 결과가 됩니다.	스크롤, 후보 소진, 조건 변경, 장애를 각각 다르게 처리했습니다.
좌표나 필터가 달라지면 이전 후보는 현재 요청과 맞지 않습니다. 반대로 페이지마다 새로 계산하면 같은 검색과 랭킹을 되풀이하게 됩니다.	정상 스크롤은 page queue만 읽고 후보가 부족할 때만 refill합니다. 요청 지문이 달라지면 후보 풀을 비우며, Redis를 사용할 수 없으면 캐시 기록을 건너뛰고 OpenSearch 후보를 메모리에서 랭킹해 응답합니다.

피드 캐시를 재사용하고, 무효화하고, 우회하는 기준

Snapshot의 상태와 요청 지문에 따라 네 경로로 나뉩니다.

정상 흐름

외부 조회

무효화 · 우회

01 요청 판별

같은 세션에서도 좌표와 필터가 달라지면 다른 요청입니다.

GET /FEED
Feed API
actor · cursor · page size

세션>
SESSION
guest · member
TTL 30분

비교>
SHA-256
Request Fingerprint
좌표 · 4축 필터

확인>
REDIS LIST
Snapshot Queue
100건 보관 · 20건씩 소비

Snapshot과 요청 조건은 지금도 유효한가?

hit · empty · changed · unavailable

H 캐시 적중

검색과 랭킹을 건너뛴니다.

LPOP 20건
정렬된 Snapshot

→

Feed page
cursor와 후보 응답

→

Global Seen
회원 24시간 · 비회원 30분

R 후보 보충

Snapshot이 비었을 때만 랭킹을 다시 계산합니다.

9×200
소스별 최대 후보

> 500
FPR

> 100
SPR · 재정렬

> 20
첫 페이지

최종 100건을 RPUSH하고 첫 20건은 바로 응답합니다.

I 조건 변경

세션과 Seen은 두고 후보 풀만 비웁니다.

지문 변경
좌표 · 필터 · 로그인 전환

→

후보 3종 삭제
Snapshot · FPR · SPR

→

다음 요청에서 보충
OpenSearch부터 재실행

F Redis 장애

캐시 기록은 건너뛰고 이번 요청에 먼저 응답합니다.

Snapshot 조회 실패
현재 요청에서 Redis 우회

→

메모리 랭킹
같은 정책으로 offset 계산

→

Feed page
캐시 · Seen 기록 없이 반환

캐시 판단 기준

정상 스크롤에서는 남은 Snapshot을 재사용하고, 조건이 바뀌면 후보 풀만 무효화합니다. Redis를 쓸 수 없을 때도 같은 랭킹을 메모리에서 계산해 응답합니다.

랭킹·캐시·검색 정책 상세

- Snapshot, FPR pool, SPR pool, user vector, global seen을 서로 다른 Redis key와 TTL로 관리했습니다.
- 9개 OpenSearch 소스를 virtual thread에서 병렬 호출하고, 한 소스가 실패하거나 timeout이 나면 해당 결과만 제외해 나머지 후보를 유지했습니다.

- Snapshot에 남은 후보가 있으면 20건만 꺼내고, 소진됐을 때만 100건짜리 랭킹 window를 다시 채웠습니다.
- seen은 노출한 페이지마다 기록하고 FPR에서 감점 신호로 사용했습니다.
- 세션과 후보 캐시는 30분, 회원 global seen은 24시간 동안 유지했습니다.
- 위치 기준 거리 가중치와 10km 반경, 좌표가 없는 광고의 보존·제외 정책을 반영했습니다.

- 메인 4축 필터를 색인과 질의 조립에 반영하고 회원·비회원에서 같은 의미로 적용했습니다.
- 유료 후보는 별도 영역으로 떼지 않고 FPR 점수 가산에 포함했습니다.
- 재게시된 같은 광고는 FPR에서 contentHash로 중복을 제거했습니다.
- 검색 인덱스 전환, read/write 계정 경계, strict mapping 이슈를 함께 다뤘습니다.

04

ENGINEERING PLATFORM · TEST SUPPORT

테스트마다 반복하던 준비 코드를 test-support로 모았습니다.

fixture와 mock 준비 방식이 프로젝트마다 달라 테스트 설정부터 다시 읽어야 했습니다. 네 가지 진입점과 다섯 가지 공통 도구·정책을 만들고, 테스트 선택 기준과 CI 실행 범위를 문서화했습니다.

공통 구성

4개 테스트 진입점 · 5개 공통 도구와 정책

모듈 구성

구축 당시 공통 진입점·도우미 구현 파일 29개

테스트

구축 당시 공통 동작 테스트 소스 43개

작성 기준

테스트 작성 원칙 제안 · 느린 테스트 실행 조건 분리

테스트 목적에 맞춰 공통 준비를 나눈 구조

진입점, 생명주기, 컨텍스트 범위, 실행 정책을 한 기준으로 묶었습니다.

공통 기반

01 · fixture·생명주기

BASE CLASS
AbstractTest

PER_CLASS · 공통 fixture

↓
FIXTURE

FixtureMonkeyFactory

fixture 생성 규칙

Mock 테스트

02 · 협력 객체 격리

BASE CLASS
AbstractMockTest

mock 생성 · 초기화

↓
EXTENSION

AutoMockExtension

주입 · reset · session 정리

MVC 테스트

03 · HTTP 요청·응답 계약

BASE CLASS
AbstractWebMvcTest

@WebMvcTest · Controller 자동 등록

↓
WEB HELPER

WrappedMockMvc

요청 · 응답 검증용 래퍼

JPA 테스트

04 · 매핑·쿼리·영속성

BASE CLASS
AbstractDataBaseTest

@DataJpaTest · Repository·Config 등록

↓
DATABASE HELPER

DB_INITIALIZER

Repository·Config Bean 등록

4개 테스트 진입점

Base · Mock · WebMvc · JPA

5개 공통 도구·정책

Fixture · Response · MockMvc · Tx Support · Slow-test

실행·작성 기준

skip.slow.test=true 제외 · 테스트 작성 원칙 제안

코드와 사용 기준

공통 도우미와 함께 테스트 진입점, mock·컨텍스트 생명주기, 별도 트랜잭션, 느린 테스트의 실행 기준을 문서화했습니다.

- 기본·mock·WebMvc·JPA 네 진입점에 FixtureMonkey와 mock 생명주기를 연결했습니다.

- Controller·Repository 자동 등록, 응답 검증, REQUIRES_NEW 트랜잭션 도우미를 공통화했습니다.
- 테스트 작성 원칙을 제안하고 느린 동시성 테스트를 skip.slow.test 설정으로 분리했습니다.

05

웍스피어 · AI ENGINEERING · AGENT LAB V0.1.0.0

에이전트·스킬을 설치 가능한 자산으로 묶고, 구조와 행동 계약을 나눠 검증했습니다.

코드 탐색·설계 검토·테스트 절차를 15개 에이전트와 32개 스킬로 나눴습니다. manifest로 의존성을 설치하고, 구조 케이스 7개·행동 케이스 10개·설치 회귀 52개를 검증했습니다.

AGENT 구성

15 agents · 32 skills

연결 구조

24 agent→skill · 11 deps

정적 계약 검증

구조 7/7 · 행동 10/10

설치·탐색

설치 52/52 · 탐색 32/32

Agent가 역할을 정하고, skill이 절차를 제공하며, KB가 판단 근거를 보탬니다.

Agent Lab이 직접 작업을 실행하지는 않습니다. 자산과 의존성을 묶어 각 런타임에 설치합니다.

— 실행 순서 — — 런타임 경계 —

ASSET REGISTRY · V0.1.0.0

manifest.txt

선택한 자산과 연결된 의존성을 한곳에서 찾습니다.

AGENTS

15

BUNDLES

14

SKILLS

32

설치된 자산을 런타임이 탐색·선택

AGENT + SKILL

Claude Code

역할과 절차를 함께 설치

01 · RUNTIME
ROUTING

Agent 선택
작업 내용으로 고르거나
이름으로 호출

참조

02 · ROLE +
KNOWLEDGE

정의와 지식 번들
.claude/agents/
SOUL · principles · KB

호출

03 · PROCEDURE

연결된 skill 실행
.claude/skills/
workflow · project
tools

SKILL LANE

Codex

32/32 skill discovery 확인

01 · DISCOVERY

Skill 탐색과 호출
/skills ·
\$skill-name

로딩

02 · WORKFLOW

절차와 참고 지식
.agents/skills/
SKILL.md · reference

실행

03 · PROJECT
CONTEXT

현재 저장소에서
작업
file · shell · git · test

SHARED EXECUTION

저장소 상태를 먼저 확인하고

코드·설정·도구를 기준으로 작업 범위를 정합니다.

OUTPUT CONTRACT

수정 근거와 검증 결과까지 남깁니다.

file:line · test result · risk · next action

01

자산 선택

agent · skill · all-*

02

연결 항목 확인

연결 skill을 재귀 탐색

03

런타임 경로에 설치

dry-run · apply · verify

04

회귀 검사

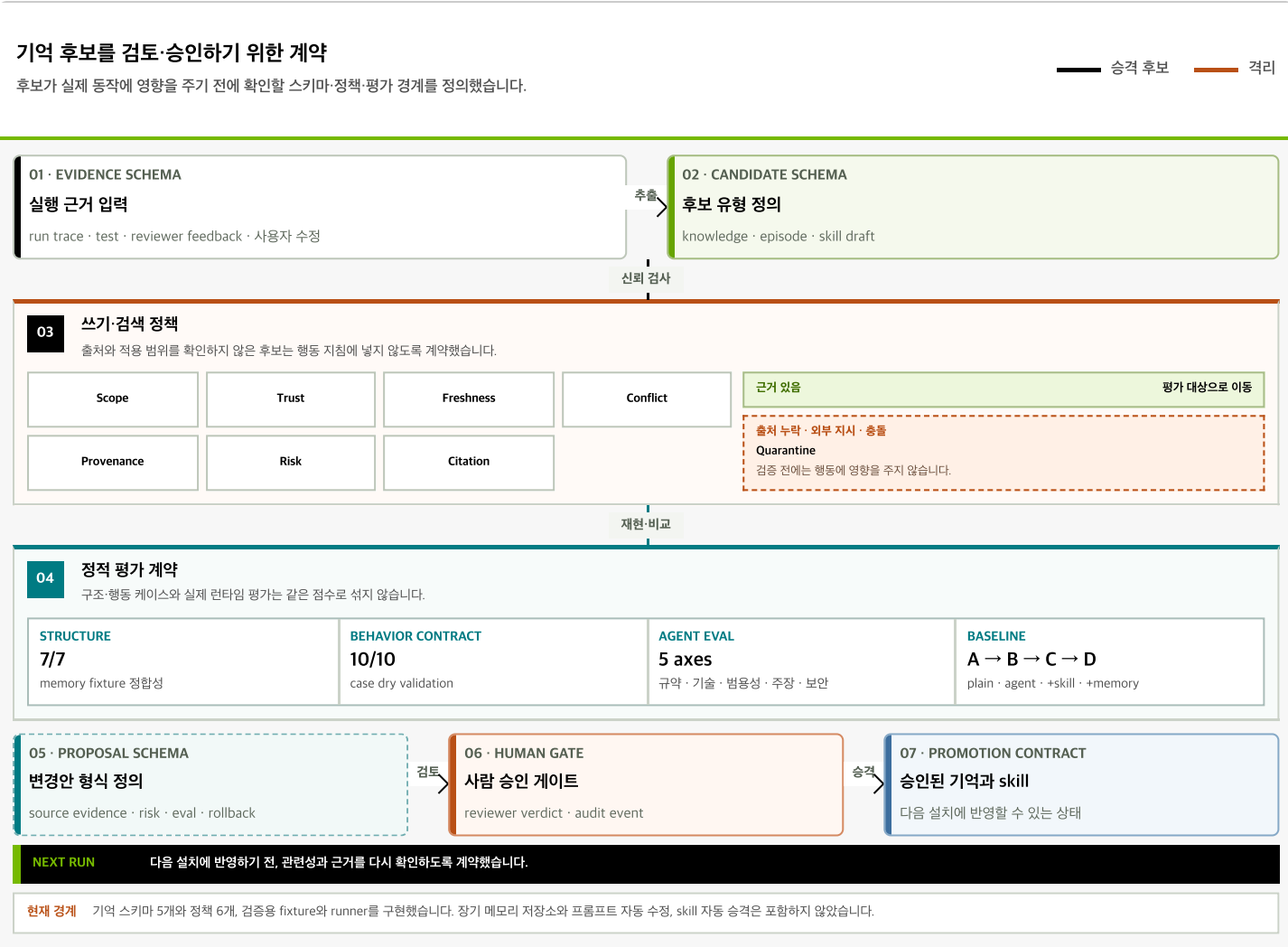
설치 52/52 · Codex 32/32

OpenCode

.opencode 경로 배치는 검사했습니다. agent 포맷 변환과 실제 선택·실행은 아직 검증하지 않았습니다.

현재 설치 범위

Claude Code에는 에이전트와 스킬을, Codex에는 스킬 32개를 설치할 수 있습니다. 메모리와 자가 수정은 스키마·정책·평가 계약으로 경계를 정의한 단계이며, 실제 반영 전에 사람 승인을 요구하는 계약을 정의했습니다.



테스트, 인증, 검색 경험이 지금의 설계 기준이 되었습니다.

실제 구현을 검증하는 테스트, 인증·인가 경계, OpenSearch 동적 쿼리를 직접 다뤘습니다. 이 경험은 이벤트 신뢰성, 배치 재시작, 피드 캐시와 검색 정책을 설계할 때의 기준이 되었습니다.

토스랩 · JANDI

2024.06 — 2025.04

커버리지 70%인데도 배포가 불안한 이유를 테스트 기준과 CI 실행 방식에서 찾았습니다. Drive-Service를 Spring Boot 2.3.8→3.3.3으로 올리며 테스트를 0→160개 이상으로 늘렸고, Mock 테스트가 놓친 JPA 변경 감지 오류는 실제 구현 테스트로 재현해 약 200만 건의 데이터를 보정했습니다.

오케스트로 · IDP / Vista

2023.09 — 2024.05

신용정보원 IDP에서 Spring Cloud API Gateway, Spring Security·JWT 인증/인가, SonarQube·Jenkins 연동 API를 개발했습니다. 하나금융그룹 Vista에서는 VM 메트릭 조회 API와 OpenSearch 동적 쿼리 분기를 구현했습니다.

즐거운 · Backend

2022.11 — 2023.02

자사 서비스의 백엔드 기능을 개발하고 운영 중 발생한 문제를 해결했습니다.

구현 과정과 판단 근거를 글·발표·오픈소스로 공개했습니다.

기술 글 156편, YouthCon 발표, Spring OSS 개선 3건으로 테스트와 데이터 복구 경험을 공유했습니다.

<u>YouthCon 2025 연사</u> '우리팀을 위한 Spring Test 만들기'에서 테스트 코드를 줄이는 기준과 팀의 그라운드 룰을 만드는 과정을 발표했습니다.	<u>기술 글 156편 · 2026.07 기준</u> Java, JPA, 테스트, SQL 튜닝을 중심으로 시행착오를 기록했습니다. 글또 10기에서는 백엔드·인프라 빌리지 준비위원과 연사로 참여했습니다.
<u>Spring OSS · 개선 제안 3건 반영</u> Kafka, Boot, Data JPA에 제안한 개선 3건이 반영됐습니다. 불변 컬렉션 처리, 하드코딩 제거, 자원 정리 로직을 개선했습니다.	<u>OpenSearch 한글 분석 플러그인</u> JavaCafe의 Elasticsearch 플러그인을 바탕으로 한·영 자판 변환, 초성·자모 검색을 OpenSearch 2.19와 Kotlin으로 옮기고 16개 예시로 확인했습니다.
<u>화면에 없던 7만+ 건의 오류</u> 응답은 정상이었지만 S3에서는 삭제되고 DB에는 남은 데이터가 6년 동안 7만 건 넘게 쌓여 있었습니다. 이를 발견하고 원인을 좁혀 간 과정을 공개했습니다.	<u>테스트로 찾은 약 200만 건</u> Mock 테스트가 놓친 JPA 변경 감지 오류를 실제 구현 테스트로 재현했습니다. 보정 스크립트를 작성해 약 200만 건의 데이터를 바로잡았습니다.